

HarvestOS

The Agentic Commerce Layer for Africa's Smallholder Farmers

Product Requirements Document — built for the AWS Communication Developer Services (CDS) Agentic AI Partner Hackathon.

Author: Mobolaji Opeyemi Bolatito (Ope) | University of Lagos, Surveying & Geoinformatics Engineering | Lagos, Nigeria

1. One-Liner

HarvestOS turns a rigid OTP/SMS pipe into a conversational commerce agent for farmers — a smallholder texts a problem (a sick crop, a cash shortfall, a delivery need) on whatever channel they already have open, and a team of specialist AI agents diagnoses, sources, finances, and fulfills it — handing the conversation off across SMS, RCS, WhatsApp and Email without the farmer ever repeating themselves.

2. Why This Wins — Mapped to the Actual Rubric

Most hackathon teams will build a single-channel chatbot demo. This is built against the five scoring categories directly, not against a generic 'cool AI idea':

Criterion	How HarvestOS scores here
Potential Value/Impact — 20%	Real, provable problem: 250M+ smallholder farmers in Sub-Saharan Africa lose 20–40% of yield to misdiagnosed crop issues and lack of access to affordable inputs/credit at the moment of need. Measurable impact story: yield saved, loans disbursed, time-to-resolution.
Creativity — 10%	Novel framing of AWS CDS: not OTP/notifications, but a full commerce loop where the *channel itself* changes based on task richness (diagnosis needs WhatsApp media; confirmation needs RCS buttons; legal/financial record needs Email).
Technical Execution — 40%	Multi-agent orchestration via Bedrock AgentCore (not a single prompt), cross-channel session state in DynamoDB, real integration of 3 CDS services (EUM SMS/RCS, EUM Social/WhatsApp, SES) plus other AWS services (Lambda, API Gateway, S3, Bedrock FMs). This is exactly where the builder's existing multi-agent systems background compounds.
Functionality — 10%	Every flow is demoable end-to-end on real phone numbers, not mocked screenshots.
Demo Presentation — 20%	The 3-minute demo shows a *complete agentic workflow with a visible channel handoff* — the single most memorable thing a judge watching 50+ videos will remember.

Your unfair advantage: you've already shipped a multi-agent agricultural intelligence system (your DSN×BCT work) and merged a fix into PyTorch core. Almost nobody else in a 347-person AWS partner hackathon field will have working multi-agent orchestration instincts walking in — most will spend half their time debugging agent handoffs you already understand.

3. Problem Statement

- Smallholder farmers overwhelmingly interact via basic phones and low-bandwidth channels (SMS/WhatsApp), not apps — any solution that requires an app install starts at a disadvantage.
- Existing agri-advisory bots are single-turn and single-channel: they diagnose but can't transact, or they sell but can't advise — so the farmer is dropped between three different tools (advisory line, agro-dealer, microfinance app).
- AWS's own brief calls this out directly: CDS today is mostly OTP and rigid scripted bots. The gap is an agentic system that carries context across channels and closes the loop into a real transaction.

4. Solution Overview

HarvestOS is an orchestrator agent (Bedrock AgentCore) sitting behind AWS CDS, backed by four specialist agents. A conversation can start on SMS, escalate to WhatsApp for a photo, resolve with an interactive RCS purchase card, and close with a formal SES email receipt — all as one continuous session, matching the exact 'prescription refill via SMS → RCS pickup → email instructions' pattern the hackathon brief describes, applied to agriculture.

5. Target Users

Persona	Need
Smallholder farmer (primary)	Owns a feature phone or entry Android. Needs diagnosis, inputs, and financing in one flow, in a channel they already use daily.
Agro-dealer / input supplier (partner)	Wants qualified demand routed to their stock without building their own tech stack.
NGO / cooperative / AWS Partner (buyer)	Wants a white-labelable platform to deploy across a farmer network and report impact to funders — this is who actually licenses HarvestOS.

6. Core User Journeys

Journey A — Diagnose → Buy → Confirm (primary demo flow)

- Farmer sends SMS: "my maize leaves are turning yellow".
- Orchestrator recognizes a diagnosis intent lacking visual evidence and replies with a WhatsApp deep-link invite to send a photo (channel escalation #1).
- Diagnosis Agent (vision + LLM) identifies likely nitrogen deficiency, explains it in plain language.
- Commerce Agent matches the nearest verified dealer with matching stock and pushes an RCS rich card: product, price, dealer distance, "Reserve for pickup" button (channel escalation #2 — richer UI for a purchase decision).
- Farmer taps to reserve. Finance Agent checks if a BNPL micro-loan is needed; if so, terms are sent and e-signed.
- SES sends a formatted receipt/invoice PDF plus a 14-day check-in reminder scheduled automatically.

Journey B — Credit-constrained purchase

- Same start as Journey A, but the farmer replies "I can't afford it right now".
- Finance Agent evaluates eligibility from conversation history + basic KYC, offers a split-payment plan.
- Formal loan agreement sent via SES; confirmation and due-date reminders sent via SMS.

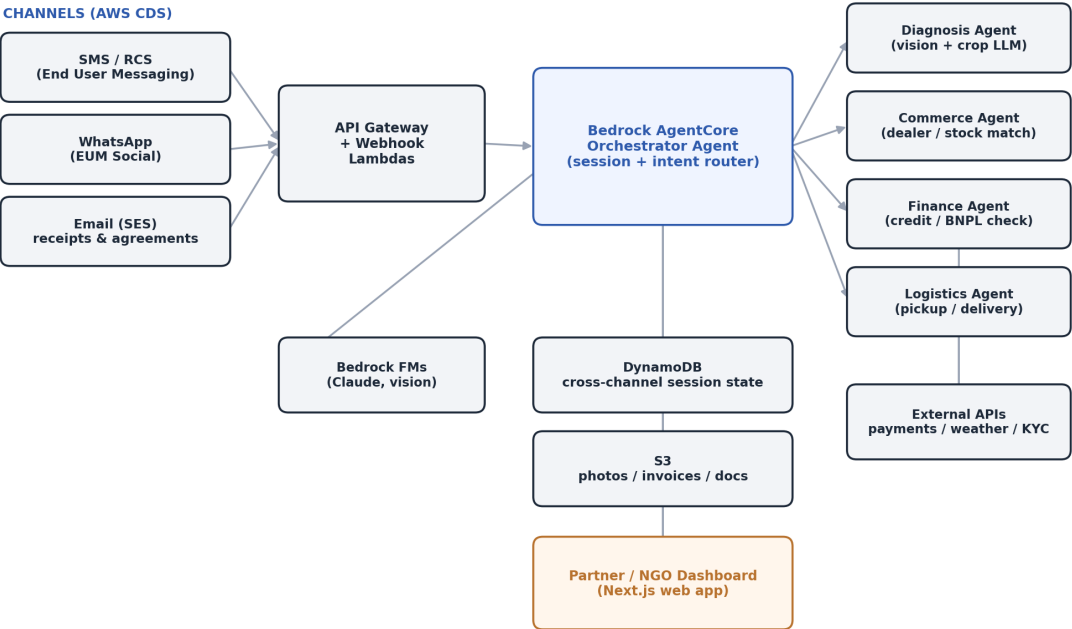
Journey C — Partner/NGO oversight

- An NGO field officer opens the HarvestOS web dashboard, sees live conversation volume, diagnosis categories trending in their region, and loan repayment health — the data story that wins the Impact/Value score with judges.

7. System Architecture

HarvestOS — Reference Architecture

Agentic omnichannel commerce, built on AWS Communication Developer Services + Bedrock AgentCore



Every hop persists to DynamoDB, so a conversation started on SMS can resolve on RCS and close on Email — the exact omnichannel continuity behavior the hackathon brief calls out.

Design principle: the Orchestrator never talks to a channel directly — every inbound and outbound message passes through the same webhook/Lambda layer, and every piece of session state lives in DynamoDB keyed by phone number. That's what makes cross-channel handoff (the brief's core ask) actually work instead of being three disconnected bots wearing one name.

8. Tech Stack

Layer	Choice
Messaging / CDS	AWS End User Messaging (SMS + RCS), AWS End User Messaging Social (WhatsApp), Amazon SES
Agentic layer	Amazon Bedrock AgentCore (orchestrator + 4 specialist sub-agents), Bedrock foundation models (text + vision)
Compute / integration	API Gateway, AWS Lambda (Node.js/TypeScript or Python webhook handlers), Step Functions (optional, for multi-step finance approvals)
Data	DynamoDB (session + conversation state), S3 (images, generated PDFs), OpenSearch (optional — dealer/stock search)
Frontend	Next.js 14 + TypeScript + Tailwind (partner dashboard + marketing site), deployed on Amplify or Vercel for the hackathon
Infra	AWS CDK (TypeScript) — the whole stack should stand up from one `cdk deploy`, which is a reproducibility point judges explicitly score

Layer	Choice
Auth	Amazon Cognito (dashboard staff logins); phone number is the identity key for farmers, no signup friction

9. AWS CDS Requirement Mapping

The rules require at least one CDS service. HarvestOS uses three, plus the recommended Bedrock AgentCore layer — this alone puts you ahead on the technical-execution rubric line about "multiple AWS CDS services... well integrated."

Service	Role in HarvestOS
EUM — SMS	Low-bandwidth entry point; works on any phone; used for the first message and reminders.
EUM — RCS	Rich interactive purchase cards, carousels, and buttons for the commerce decision moment.
EUM Social — WhatsApp	Photo upload for crop diagnosis (also targets the Meta "Best of WhatsApp" \$10,000 prize).
SES	Formal receipts, loan agreements, and any document that needs to be retained/printed.

10. Repository Structure

One monorepo, cleanly separated so backend, frontend, infra and docs can be built and demoed independently:

```
harvestos/
|-- backend/
|   |-- agents/
|   |   |-- orchestrator/          # intent routing, session control
|   |   |-- diagnosis_agent/      # vision + crop-health reasoning
|   |   |-- commerce_agent/      # dealer/stock matching
|   |   |-- finance_agent/       # credit/BNPL eligibility
|   |   |-- logistics_agent/     # pickup/delivery coordination
|   |-- channels/
|   |   |-- sms_rcs_handler/      # EUM webhook + outbound
|   |   |-- whatsapp_handler/    # EUM Social webhook + outbound
|   |   |-- email_handler/       # SES send + templates
|   |-- shared/                  # session store, models, utils
|   |-- tests/
|-- frontend/
|   |-- landing/                  # marketing site (Next.js)
|   |-- dashboard/               # partner/NGO ops dashboard (Next.js)
|-- infra/
|   |-- cdk/                      # AWS CDK app - one command deploy
|-- docs/
|   |-- PRD.pdf                   # this document
|   |-- architecture.png
|   |-- demo-script.md
|-- README.md                     # setup, run, and demo instructions
```

11. MVP Scope for the Hackathon Window (~5 weeks)

Window	Deliverables
Week 1	CDK skeleton up; SMS + WhatsApp webhooks live; DynamoDB session schema; orchestrator agent routing to a stub Diagnosis Agent.
Week 2	Diagnosis Agent wired to Bedrock vision model on real crop photos; Commerce Agent with a seeded dealer/stock dataset; RCS rich cards working end-to-end.
Week 3	Finance Agent (rule-based eligibility is fine for MVP — no need for a real credit bureau integration); SES receipt + agreement templates; reminder scheduling.
Week 4	Partner dashboard (Next.js): live conversation feed, diagnosis category breakdown, loan health widget. Landing page. Polish the three demo journeys until they're bulletproof.
Week 5	Record the 3-minute demo video, write the architecture diagram + text description, create the ACE opportunity, submit.

12. Frontend Design Direction — "\$10B SaaS," Not "AI Demo"

The instruction was explicit: no generic AI-slop UI (no purple gradient blobs, no default shadow hero with a floating chat bubble, no stock "robot hand touching leaf" imagery). Design direction for both the landing page and the partner dashboard:

- **Typography-led, not illustration-led.** One strong serif or high-contrast grotesque for headlines (think Söhne, General Sans, or a well-set Inter with tight tracking), restrained color — near-black ink, one accent (a deep agricultural green or an earthy ochre, not default indigo/purple).
- **Real data, not mock hero art.** Landing page hero should show an actual product screen — the dashboard's live conversation feed or a real RCS card mockup on a phone frame — not an abstract graphic.
- **Dashboard reads like Linear or Stripe's dashboard, not a hackathon admin panel:** dense but calm data tables, quiet borders instead of heavy shadows, one clear primary action per screen.
- **Motion is restrained** — used to clarify state changes (a message moving between channels in the live feed), never decorative.
- Build the landing page and dashboard as separate Next.js apps in /frontend so either can be iterated or demoed independently without touching the other.

13. Demo Video Script Outline (target: 3 minutes)

Timestamp	Beat
0:00–0:20	Problem in one sentence, over a real (not stock) image or your own dashboard chart of the yield-loss stat.
0:20–1:40	Live Journey A end-to-end on a real phone: SMS → WhatsApp photo → diagnosis → RCS card → tap to reserve → SES receipt arriving. This is the section judges will remember — do not cut away from the phone screen.
1:40–2:10	10-second cut to the partner dashboard showing the same conversation appear live, plus the impact metrics.
2:10–2:40	Architecture diagram, 15 seconds, narrated plainly: which AWS CDS + Bedrock services, and why the hand-off is agentic rather than scripted.
2:40–3:00	Close on the ACE opportunity / GTM angle — this is a partner offering an NGO or agri-fintech can license, not just a demo.

14. Risks & Mitigations

Risk	Mitigation
WhatsApp/RCS provisioning delays	Request AWS CDS sandbox numbers and Kiro/credit codes on day 1 — these are first-come, first-served per the rules.
Vision diagnosis accuracy on live demo	Curate a small, tested set of real crop photos you've verified produce a clean diagnosis; don't improvise with an unseen photo live.
Scope creep across 4 agents	Build the Orchestrator + Diagnosis + Commerce path first — that alone is a complete, demoable story. Finance and Logistics agents can be thinner for MVP.
Judges can't tell it's really agentic vs. a scripted flow	Show the AgentCore trace/reasoning log for at least one decision on screen during the demo — makes the orchestration visible, not just implied.

15. Success Metrics (for the pitch)

- Time from first message to resolved outcome (diagnosis+purchase or diagnosis+financing) — target under 5 minutes in demo.
- Number of distinct AWS CDS channels a single session traverses (target: 3 — SMS, WhatsApp, RCS, closing via SES).
- Simulated/pilot cohort: % of flagged nitrogen-deficiency cases where a purchase or financing action was completed.

16. Submission Checklist

- GitHub/GitLab repo (public with OSS license visible in About, or private + shared with testing@devpost.com and aws-cds-partner@amazon.com).
- Architecture diagram (use /docs/architecture.png as the base) + text description.
- ~3-minute demo video following the script above.
- URL to deployed project or clear run instructions.
- AWS Partner Central ACE opportunity created with campaign code "AWS CDS Agentic AI Hackathon -Sept. 2026."
- WhatsApp usage write-up if entering the Meta "Best of WhatsApp" track.

HarvestOS — Product Requirements Document. Prepared for the AWS Communication Developer Services (CDS) Agentic AI Partner Hackathon (deadline Oct 28, 2026).